Lesson 7 Teacher's Key: On the Move

Unreal Engine (optional advanced track) - Instructor Reference Only

VIDEO GAME CREATION

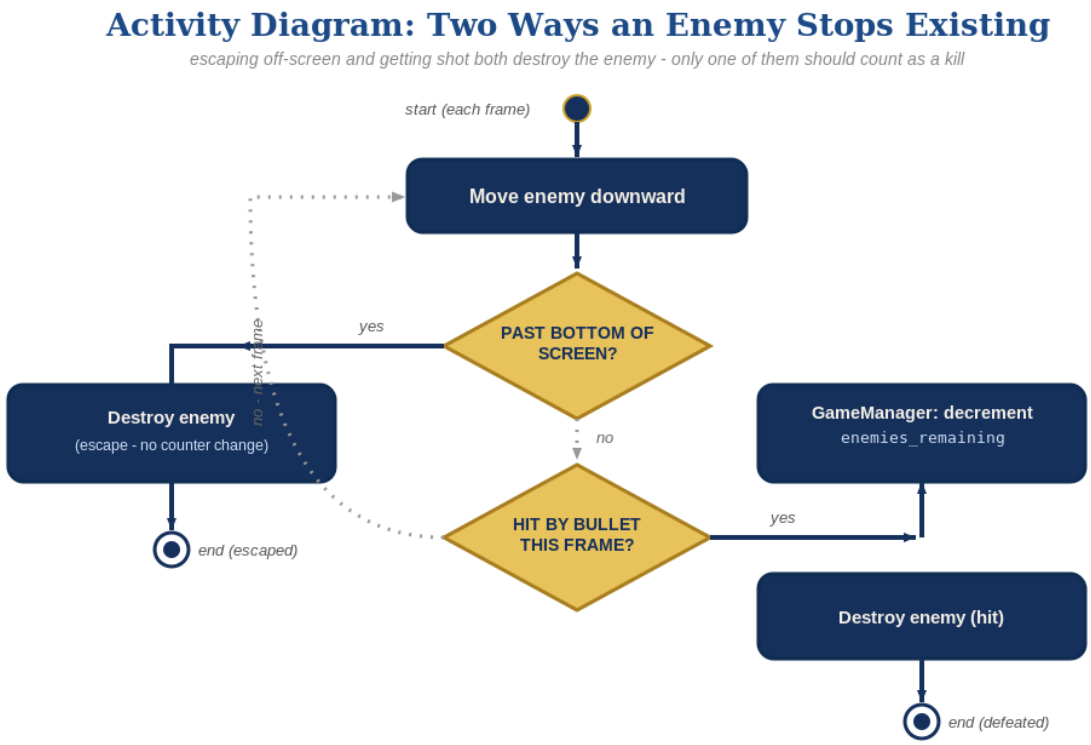
What This Lesson Teaches

The same escape-vs-hit distinction as Godot/Unity, expressed as a plain Destroy Actor node (called from the enemy Blueprint's own Tick-driven escape Branch) vs. the OnEnemyDestroyed dispatcher (broadcast only from the successful-Cast branch of the overlap event).

The Bug Being Fixed

No bugfix needed for Unreal - Lesson 6's design already broadcast OnEnemyDestroyed exclusively from the overlap event's successful-Cast branch, and this lesson's new Tick-driven escape logic calls a plain Destroy Actor with no dispatcher broadcast, same pattern as Godot/Unity.

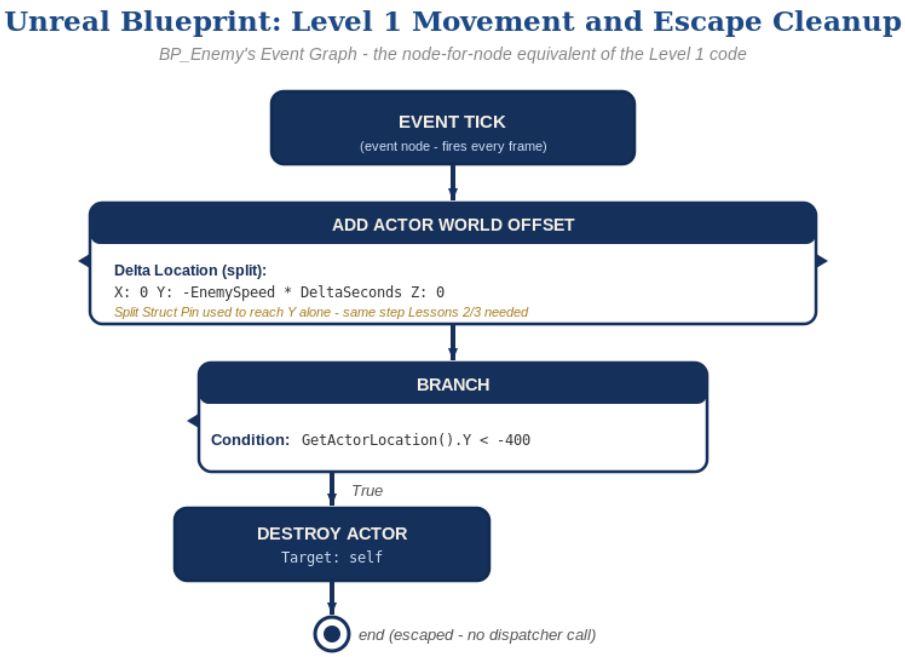
Level 1 - Full Worked Solution



These two checks look similar - both end with "the enemy is gone" - but only the hit check is allowed to touch enemies_remaining. Attaching the counter to a generic "was destroyed" hook instead of this specific hit check is exactly the bug GameMaker and 001 Game Creator have to fix this lesson.

The general escape-vs-hit distinction behind this week's exercise - the same shape in every engine, whatever the exact code or blocks look like.

- Event Tick drives both the movement (Add Actor World Offset along -Y, Delta Location's pin Split Struct Pinned first so -EnemySpeed*DeltaSeconds can be wired into just Y - same step Lessons 2 and 3 needed) and the escape check (a Branch comparing GetActorLocation().Y against a threshold) every frame - Blueprint's Tick behaves the same as Godot's _process or Unity's Update.
- The escape Branch's True case calls a plain Destroy Actor node - no connection to OnEnemyDestroyed at all.
- DeltaSeconds scaling on the movement offset keeps it frame-rate independent, consistent with patterns taught since Lesson 2.



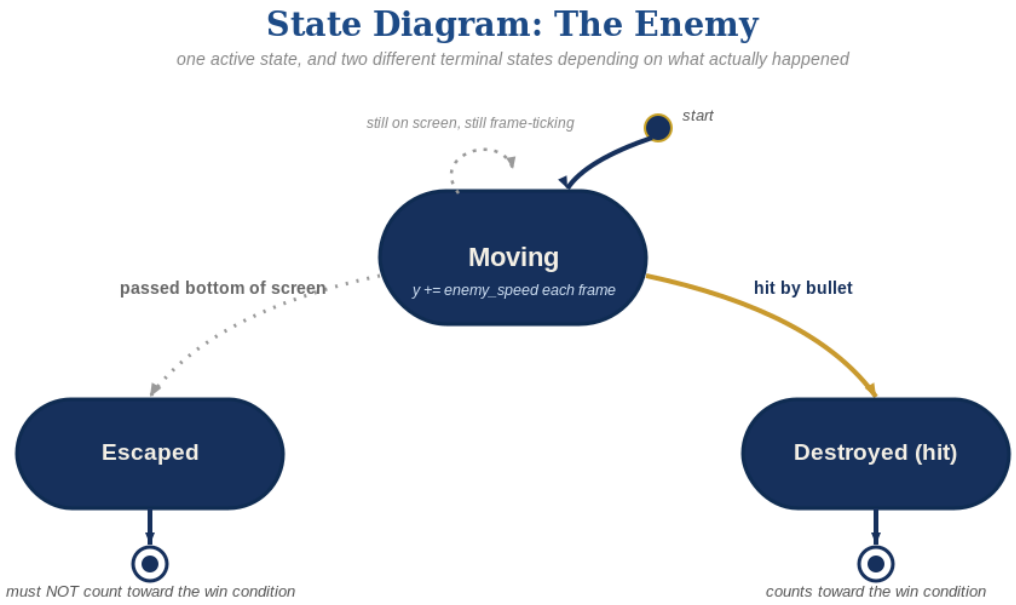
Compare this graph to the overlap event's successful-Cast branch in the Key: only that separate event ever calls OnEnemyDestroyed, which now feeds BP_GameManager's HandleEnemyDestroyed event - Event Tick above never touches it.

BP_GameManager was introduced in Lesson 6 - a placed Actor the Level Blueprint holds a reference to, not the Level Blueprint itself.

BP_Enemy's Event Tick graph, in Unreal Blueprint - the node-for-node equivalent of the Level 1 code.

Why This Structure

- Same principle as Godot/Unity: the dispatcher broadcast is reserved exclusively for the overlap event's successful-Cast branch (an actual hit), and the escape-cleanup Branch has no reason to know the dispatcher exists.
- This is the third engine in a row (after Godot and Unity) where the Lesson 6 structure already prevents this bug - worth using as a three-way comparison to reinforce that 'attach state changes to the specific event, not the generic one' is a cross-engine principle, not an engine-specific trick.



Once an enemy leaves Moving, it never comes back - both destinations are final. The whole trick this lesson teaches is making sure your code can tell these two final states apart, since only one of them is a kill.

The enemy's two possible endings this week - only one of them should count as a kill.

Level 2 / Level 3 Talking Points

- For students on the C++ track: point out that the same distinction (Tick calling a plain Destroy() vs. OnOverlap calling Broadcast()) holds identically in code - the principle isn't Blueprint-specific.
- Ask advanced students: what's the Unreal-specific risk of doing heavy logic in Tick for many enemies at once (performance, since Tick runs every frame for every actor) - a good preview of optimization topics for students continuing past this course.

Common Student Mistakes

- Wiring the escape Branch's True case to Call OnEnemyDestroyed 'to keep things consistent' - reintroduces the exact bug this lesson is designed to avoid.
- Forgetting DeltaSeconds in the Add Actor World Offset node (frame-rate-dependent movement).
- Using the wrong sign on the Y offset (positive instead of negative, given this course's 'Y+ is up' convention) and having enemies move upward off the top instead of downward.